

Лабораторная 5. Бинарное дерево поиска

Для чего нужно дерево

В списке поиск неизвестного значения может потребовать просмотра всех элементов. Бинарное дерево поиска, BST, использует порядок ключей, чтобы на каждом шаге выбрать только одно направление. В этой работе ключи уникальны: повторная вставка существующего ключа возвращает False и не создаёт узел. Требуются вставка, поиск, удаление, четыре обхода, высота и сравнение со списком.[1]

У каждого узла есть key, left и right. Инвариант BST относится ко всему поддереву: все ключи слева строго меньше ключа узла, все справа строго больше. Проверить только непосредственных детей недостаточно. Например, у корня 10 левый ребёнок 5 может иметь правого потомка 12: локальное отношение $5 < 12$ верно, но 12 нарушает ограничение левого поддерева корня.

Поиск и вставка по шагам

Пусть последовательно вставлены [8,3,10,1,6,14,4,7,13]. Число 6 ищется по пути $8 \rightarrow 3 \rightarrow 6$: оно меньше 8, но больше 3. Число 5 пошло бы $8 \rightarrow 3 \rightarrow 6 \rightarrow 4 \rightarrow$ пустая правая ссылка. Достижение пустой ссылки означает, что ключ отсутствует.

Вставка проходит тот же путь и создаёт новый лист на месте пустой ссылки. Уже существующие отношения порядка не меняются. Поиск, вставка и удаление реализованы итеративно: глубина дерева не превращается в глубину стека этих операций.

Три случая удаления

Лист. У узла нет детей. Ссылка родителя на него заменяется на None. Если удалялся единственный корень, дерево становится пустым.

Один ребёнок. Родитель начинает ссылаться непосредственно на ребёнка удалённого узла. Весь его диапазон ключей уже допустим на этом месте. При удалении корня ребёнок становится новым корнем.

Два ребёнка. Нельзя просто оставить одного ребёнка: потеряется другое поддерево. Ищем преемника — минимальный ключ в правом поддереве. Он больше всех ключей слева и не больше остальных ключей справа. Копируем его ключ в удаляемый узел, затем физически удаляем узел преемника. У преемника нет левого ребёнка, поэтому осталось только одно из первых двух действий.[2]

В приведённом примере при удалении 8 преемник равен 10. Если удалить 3, преемник — 4: нужно спуститься в правое поддерево к 6 и затем налево. Особый случай — преемник является непосредственным правым ребёнком; код отдельно сохраняет его родителя, поэтому работает и в этой ситуации.

Четыре порядка обхода

Обход	Порядок действий	Результат на дереве из примера
in-order	Лево, узел, право	1,3,4,6,7,8,10,13,14
pre-order	Узел, лево, право	8,3,1,6,4,7,10,14,13
post-order	Лево, право, узел	1,4,7,6,3,13,14,10,8
level-order	По уровням	8,3,10,1,6,14,4,7,13

In-order выдаёт ключи по возрастанию и удобен для проверки BST. Pre-order обрабатывает родителя до потомков; при сериализации произвольного дерева потребуются также маркеры пустых ссылок или другая информация о форме. Post-order полезен, когда результат родителя зависит от результатов детей. Level-order использует очередь: сначала извлекается самый ранний добавленный узел, затем в очередь ставятся его дети.

Первые три обхода реализованы рекурсивно по требованию лабораторной. Их время $O(n)$, дополнительный стек $O(h)$, а возвращаемый список результатов занимает $O(n)$. На вырожденном дереве большой глубины рекурсивный обход может получить RecursionError. Увеличивать лимит без необходимости не следует; в прикладном варианте обход заменяется явным стеком. Итеративные validate, height и операции дерева этого ограничения не имеют.

Высота определяет время

Высота h здесь измеряется числом рёбер в самом длинном пути от корня до листа. Пустое дерево имеет $h=-1$, один узел — $h=0$. Поиск посещает максимум $h+1$ узлов, поэтому время $O(h+1)$. Для непустого большого дерева обычно пишут $O(h)$.

При случайном порядке вставки различных ключей ожидаемая высота имеет порядок $O(\log n)$. Это вероятностное свойство, а не гарантия для любого входа. При вставке 1,2,3,... каждый новый узел становится правым ребёнком предыдущего: $h=n-1$ и операции становятся $O(n)$. Построение такого дерева последовательными вставками требует $\Theta(n^2)$ времени.

AVL-дерево контролирует разницу высот детей: не больше 1 по модулю. После изменения выполняются повороты. Если $N(h)$ — минимальное число узлов высоты h , то $N(h)=1+N(h-1)+N(h-2)$, $N(-1)=0$, $N(0)=1$. Следовательно, $N(h)=F(h+3)-1$ и $h \leq 1,4405 \cdot \log_2 n + O(1)$. В КИМ приведена сокращённая оценка $1,44 \cdot \log_2 n$; точные границы зависят от определения высоты и добавочной константы. Нельзя понимать округлённый коэффициент как точное неравенство для каждого малого n .

Сама реализация этой лабораторной — обычное BST, она не балансируется. AVL объясняется как способ получить гарантированную логарифмическую высоту, а не приписывается данному коду.

Как проверяется корректность

Метод validate переносит вниз нижнюю и верхнюю допустимые границы каждого ключа, считает узлы и обнаруживает повторное посещение одного объекта. Это проверяет весь диапазон поддерева и возможные циклы ссылок. Дополнительно in-order сравнивается с sorted эталонного set.

Тесты включают 4000 случайных команд insert/search/delete. Сравниваются не только ключи, но и возвращаемые значения: True при реальном изменении, False при повторной вставке или удалении отсутствующего ключа. Отдельные случаи проверяют удаление листа, корня, узла с одним ребёнком и узла с двумя детьми.

```
python -m pytest labs/lab05 -q
python -m labs.lab05.experiment --variant 15
```

На пяти размерах 1000...100000 дерево строится из случайных данных варианта 15. Затем одинаковые успешные и неуспешные запросы выполняются в BST и списке. Время построения не включено в время поиска; при выборе структуры для единственного запроса эту стоимость нужно учитывать отдельно. Дубликаты в массиве схлопываются в дереве, поэтому в CSV высот указано также число уникальных ключей.

Вопросы на защите и ответы

- 1. Чем отличаются обходы?** Местом обработки текущего узла относительно обхода детей. In-order сортирует ключи BST, pre-order начинает с родителя, post-order заканчивает им, level-order идёт по расстоянию от корня.
- 2. Почему преемник подходит для удаления узла с двумя детьми?** Это наименьший ключ правого поддерева. Он остаётся больше всех левых ключей и меньше всех остальных правых. Его собственное удаление проще, поскольку левого ребёнка нет.
- 3. От чего зависит сложность операций?** От высоты и длины пути поиска, а не только от числа узлов. При $h \approx \log n$ поиск быстрый; при $h = n - 1$ он линейный.
- 4. Как AVL обеспечивает $O(\log n)$?** Локальное ограничение баланса не позволяет длинной цепочке появиться без большого числа узлов. Минимальное число узлов растёт как числа Фибоначчи; обратная зависимость высоты от n логарифмическая.
- 5. Почему простой порядок детей недостаточен для проверки?** Ограничения наследуются от всех предков. Потомок слева от 10 должен быть меньше 10, даже если его непосредственный родитель намного меньше.
- 6. Что происходит при повторной вставке?** Метод возвращает False, размер не увеличивается. Иначе пришлось бы отдельно определить размещение повторов или счётчик частот в узле.
- 7. Чем отличается len от count_nodes?** len читает поддерживаемый счётчик за $O(1)$. count_nodes независимо обходит дерево за $O(n)$; это удобно для проверки счётчика.
- 8. Почему дерево может проиграть списку на малых n ?** Собственные Python-узлы и переходы по ссылкам имеют накладные расходы, а поиск in по списку реализован в C. Оценка роста не гарантирует выигрыш на каждом маленьком входе.

Источники

[1] МТУСИ. КИМ-05, 2026.

[2] Robert Sedgewick, Kevin Wayne. Binary Search Trees, материалы авторов Algorithms, 4th edition.

Отчёт по ЛР 5. Бинарное дерево поиска

Цель и выполненная работа

Реализованы алгоритмы и проверки, описанные в конспекте. Эксперимент выполнен на данных варианта 15, seed 45. В таблицах приведены фактические результаты запуска; время указано в секундах, если не оговорено иное.

Условия и методика

CPU: 12th Gen Intel(R) Core(TM) i7-12650H. ОС: Windows-11-10.0.26200-SP0. Python 3.12.12, NumPy 2.5.3, Matplotlib 3.11.1. Дата и время UTC: 2026-09-10T18:29:16.963305+00:00.

Один прогрев и пять повторов на точку, итог — медиана `time.perf_counter`. Ввод, генерация и подготовка свежего состояния исключены из таймера. Фоновые процессы и питание не контролировались; задержки рабочего компьютера могут влиять на отдельные точки. Контрольные суммы данных находятся в `environment.json`.

Результаты и их объяснение

Порядок	Вставок	Уникальных	Высота
random	1000	1000	23
random	3000	2994	25
random	10000	9981	31
random	30000	29780	37
random	100000	97522	40
sorted	16	16	15
sorted	64	64	63
sorted	256	256	255
sorted	512	512	511

Запрос	Структура	с/запрос
hit	BST	1.68437e-06
hit	list.in	0.000511542
miss	BST	1.34531e-06
miss	list.in	0.000950916

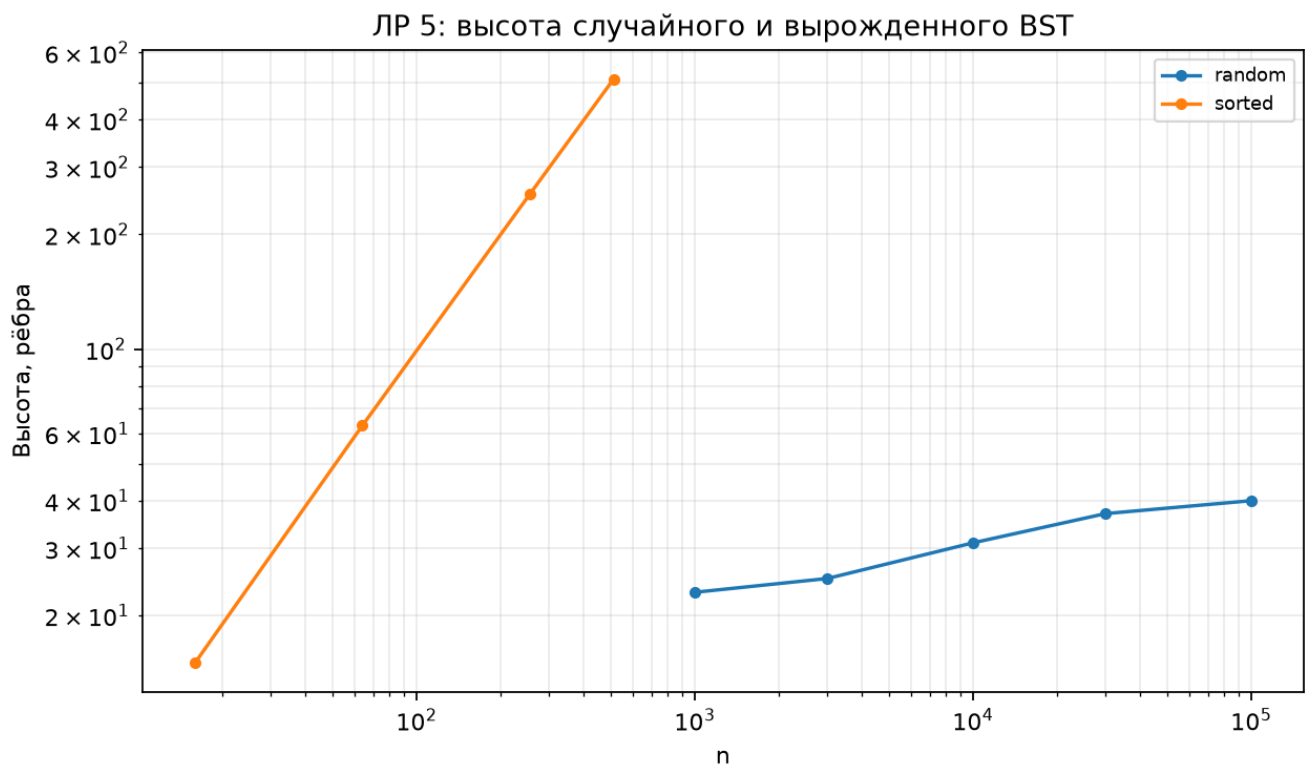
При 100000 исходных значений случайное дерево имеет высоту 40; число уникальных узлов меньше из-за повторов. Упорядоченный контрольный вход даёт $h=n-1$. Поиск измеряется после построения; стоимость подготовки структуры в эти значения не входит. Для случайного BST логарифмическая высота является ожидаемым свойством, а не гарантией AVL.

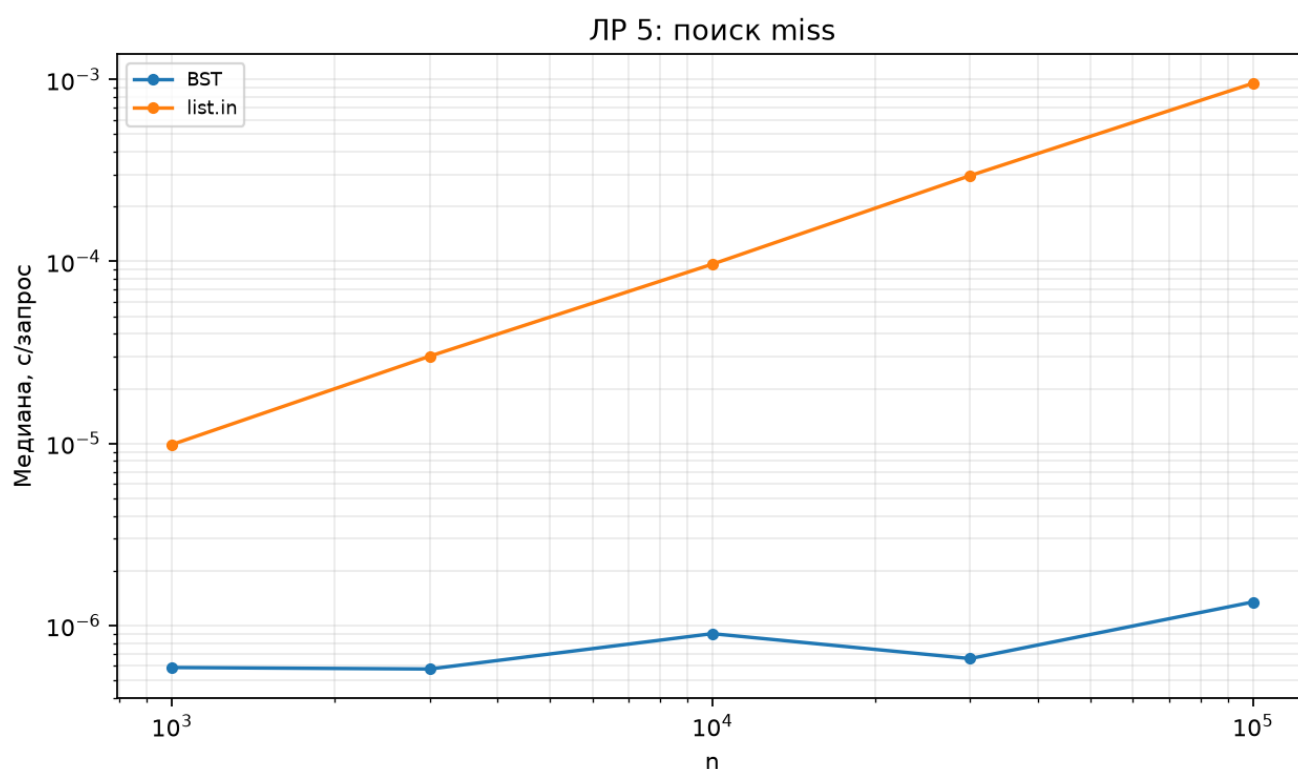
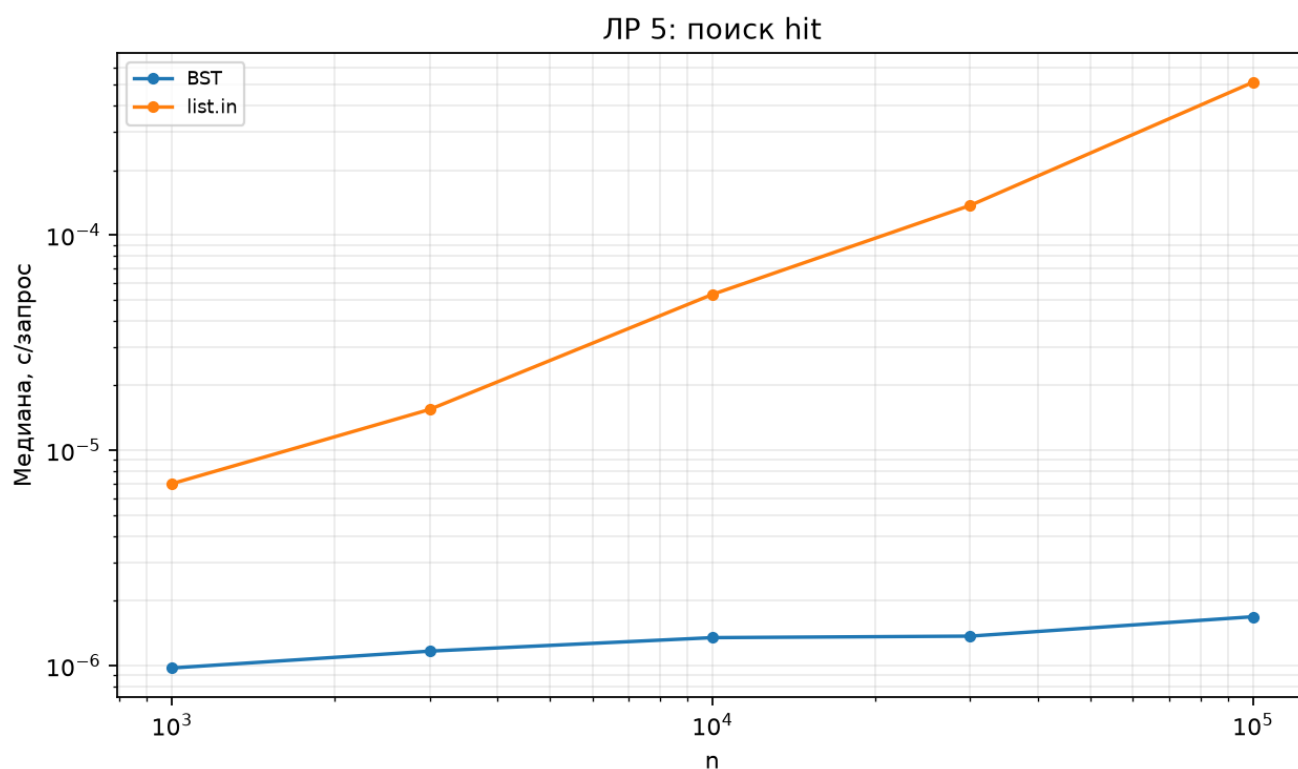
Проверка и вывод о применимости

Проверки находятся в `test_lab.py`. Запуск: `python -m pytest labs/lab05 -q`. Применены граничные случаи, инварианты и независимые эталоны; состав тестов подробно разобран в конспекте. Проверки всего комплекта также сохранены в `verification.txt` в корне репозитория.

Вывод: принять для учебного использования в рамках явно описанных контрактов. Измерения подтверждают основные тенденции роста; ограничения реализаций и сравнения указаны выше и в конспекте. Автоматическая проверка не оценивает готовность к устной защите.

Графики





Исходные данные отчёта

CSV содержит медиану и все пять длительностей trial_1...trial_5. Вспомогательные таблицы без времени содержат точные счётчики или свойства структуры.

- heights.csv
- search.csv

- Паспорт окружения и контрольные суммы